Technica in practice

A short guide to reliable retries

Prose, code, and technical notes

Technical writing moves between explanation and implementation. This sample uses a small retry policy to demonstrate Technica's typography, semantic prose, code listings, terminal sessions, and callouts.

1 Explain the idea

A **retry budget** limits how often an operation may be attempted again. The delay between attempts should grow, while the total amount of work remains bounded. A simple policy starts at 100 milliseconds and doubles the delay up to a fixed ceiling:

$$d_n = \min\bigl(100 \cdot 2^n,\ 1600\bigr)\ \text{ms}, \qquad n = 0, 1, 2, \ldots$$

Here, $n = 0$ is the first retry. Five scheduled retries wait for a total of $100 + 200 + 400 + 800 + 1600 = 3100$ milliseconds, excluding the time spent performing the operation itself.

1.1 Give names to the moving parts

Store the policy in `config/retry.conf`. Use an API such as `retry_delay` to calculate each delay, and inspect the sample with `cat config/retry.conf`. In an editor, press CTRL + S to save a change.

Note

> The initial request is not a retry. A budget of five retries permits up to six attempts in total.

Explanation

> Increasing the delay gives a busy service time to recover. The ceiling keeps individual waits bounded; a separate budget bounds the number of attempts.

2 Show the implementation

The C++ example below calculates delays without sleeping or making network requests. Applying the cap at each step keeps the arithmetic bounded.

The return type is `std::chrono::milliseconds`.

retry.cpp

```cpp
#include <algorithm>
#include <chrono>
#include <iostream>

using namespace std::chrono_literals;

std::chrono::milliseconds retry_delay(unsigned retry) {
    auto delay = 100ms;
    constexpr auto cap = 1600ms;
    while (retry > 0 && delay < cap) {
        delay = std::min(delay * 2, cap);
        --retry;
    }
    return delay;
}

int main() {
    for (unsigned n = 0; n < 5; ++n) {
        std::cout << retry_delay(n).count() << " ms\n";
    }
}
```

Build and run

```
$ clang++ -std=c++17 retry.cpp -o retry
$ ./retry
100 ms
200 ms
400 ms
800 ms
1600 ms
```

Warning

> A timeout does not prove that an operation failed. Retrying a payment or a write can duplicate its effect unless the operation is safe to repeat.

3 Connect the details

Code block 1 on page 2 implements the delay rule. Keep the policy visible alongside the code so readers can compare the configuration with its behavior.

config/retry.conf

```
initial_delay_ms = 100
maximum_delay_ms = 1600
maximum_retries = 5
```

3.1 Review the policy

1. Decide which failures are safe to retry.
2. Set both an attempt limit and an overall time budget.
3. Record attempts so repeated failures are observable.

Advice

For many clients, add random variation to the delay. Otherwise, clients that fail together may also retry together and create another burst of traffic.

Design rule

Keep the timing calculation separate from the operation that waits. A pure calculation is easier to inspect and test with exact expected values.

3.2 Use the style in your own document

The source of this sample uses the standard article class and the default page dimensions. Start with the following preamble; choose the dark theme by changing `light` to `dark`.

Minimal preamble

```
\documentclass{article}
\usepackage[theme=light]{technica}
\usepackage{hyperref}
```

The ordinary prose commands, code environments, and callouts all follow the selected theme. Hyperlinks and references work when the document loads `hyperref`; compile twice to resolve page and block references.